

刘宇轩 2025112195

第8讲 Python程序实践一

示例1：判断4位整数是否为回文数

```
def palindrome():
    n = input("请输入一个四位数:")
    n = int(n)
    a = n // 1000
    b = n // 100 % 10
    c = n // 10 % 10
    d = n % 10
    m = d * 1000 + c * 100 + b * 10 + a
    result = (n == m)
    print("回文数的判断结果是:", result)

if __name__ == '__main__':
    palindrome()
```

程序逻辑解析：

该程序通过数学运算分离四位数的各个数位，然后逆序重组为新数字，最后比较原数字与重组数字是否相等来判断是否为回文数。

程序语法要点：

- **整除运算 `//`**：获取整数商，用于分离高位数字
- **连续运算 `n // 100 % 10`**：先整除再取余，分离中间数位
- **布尔表达式赋值 `result = (n == m)`**：比较结果直接存储为布尔值

示例2：每天进步/退步效率计算

```
def progress():
    ex = float(input("输入每天的提升效率:"))
    dayup = (1.0 + ex) ** 365
    daydown = (1.0 - ex) ** 365
    print("向上:{:.2f}, 向下:{:.2f}".format(dayup, daydown))

if __name__ == '__main__':
    progress()
```

程序逻辑解析：

通过复利公式计算每天固定效率提升/下降后，一年内的累计效果，展示微小变化的长期影响。

程序语法要点：

- **幂运算** `**`：计算复利增长
 - **浮点格式化** `{:.2f}`：控制小数点后两位输出
-

示例3：输出斐波那契数列前n项

```
def fibonacci(n):  
    a = 0  
    b = 1  
    print(a, b)  
    for _ in range(n):  
        a, b = b, a + b  
        print(b)  
    return
```

程序逻辑解析：

使用迭代方法生成斐波那契数列，通过变量交换实现前两项相加得到下一项。

程序语法要点：

- **多重赋值** `a, b = b, a + b`：同时更新两个变量
 - **循环占位符** `for _ in range(n)`：忽略循环计数
-

示例4：判断整数是否为回文数

```
def is_palindrome(num):  
    if num < 0:  
        return False  
    original = num  
    reversed_num = 0  
    while num > 0:  
        reversed_num = reversed_num * 10 + num % 10  
        num = num // 10  
    return original == reversed_num  
  
# 测试  
print(is_palindrome(121))  
print(is_palindrome(-121))
```

程序逻辑解析：

通过数学运算将数字反转，比较原数字与反转后数字是否相等来判断回文数。

程序语法要点：

- **数字反转算法** `reversed_num = reversed_num * 10 + num % 10`
- **边界处理** `if num < 0: return False`：处理负数情况

示例5：计算n的阶乘

(循环版)

```
def factorial(n):
    if n < 0:
        raise ValueError("n 不能为负数")
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
# 测试
print(factorial(5))
```

(递归版)

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
# 测试
print(factorial(5))
```

程序逻辑解析：

循环版通过迭代累乘计算阶乘，递归版通过函数自调用分解问题。

程序语法要点：

- **递归基线条件** `if n == 0: return 1`
- **异常抛出** `raise ValueError("n 不能为负数")`

示例6：反转32位有符号整数

```
def reverse(x):
    sign = -1 if x < 0 else 1
    x = abs(x)
    reversed_x = int(str(x)[::-1])
    if reversed_x > 2 ** 31 - 1:
        return 0
    return sign * reversed_x
# 测试
print(reverse(123))
print(reverse(-123))
print(reverse(120))
```

程序逻辑解析：

将数字转为字符串反转后再转回整数，处理符号和整数溢出边界条件。

程序语法要点：

- 字符串反转 `str(x)[::-1]`
- 整数范围检查 `if reversed_x > 2 ** 31 - 1`

示例7：检查正整数是否为完全平方数

```
def is_perfect_square(n):  
    if n < 0:  
        return False  
    root = int(n ** 0.5)  
    return root * root == n  
# 测试  
print(is_perfect_square(16))  
print(is_perfect_square(15))
```

程序逻辑解析：

计算平方根后取整，验证平方是否等于原数来判断完全平方数。

程序语法要点：

- 平方根计算 `n ** 0.5`
- 整数验证 `return root * root == n`

示例8：计算最大公约数和最小公倍数

```
def gcd(a, b):  
    while b:  
        a, b = b, a % b  
    return a  
def lcm(a, b):  
    return abs(a * b) // gcd(a, b)  
# 测试  
print(lcm(12, 18))
```

程序逻辑解析：

使用欧几里得算法辗转相除求最大公约数，利用公式计算最小公倍数。

程序语法要点：

- 辗转相除法 `a, b = b, a % b`
- 公式应用 `abs(a * b) // gcd(a, b)`

示例9：整数转换为二进制字符串

```
def int_to_binary(n):
    if n == 0:
        return "0"
    binary = ""
    is_negative = False
    if n < 0:
        is_negative = True
        n = -n
    while n > 0:
        binary = str(n % 2) + binary
        n = n // 2
    return "-" + binary if is_negative else binary

# 测试
print(int_to_binary(10))
```

程序逻辑解析：

通过连续除以2取余数的方法构建二进制表示，处理负数和零的特殊情况。

程序语法要点：

- 二进制构建 `binary = str(n % 2) + binary`
- 负数处理 `is_negative = True; n = -n`

示例10：判断字符串是否包含子串

```
def contains_substring(s, substr):
    substr_len = len(substr)
    s_len = len(s)
    if substr_len == 0:
        return True
    if substr_len > s_len:
        return False
    for i in range(s_len - substr_len + 1):
        if s[i:i+substr_len] == substr:
            return True
    return False

# 测试
print(contains_substring("Hello World", "World"))
```

程序逻辑解析：

通过滑动窗口遍历主字符串，逐个位置检查是否匹配目标子串。

程序语法要点:

- 滑动窗口 `s[i:i+substr_len] == substr`
 - 边界条件 `s_len - substr_len + 1`
-

示例11: 返回1到n之间能被3或5整除的数

```
def multiples_of_3_or_5(n):  
    return [num for num in range(1, n+1) if num % 3 == 0 or num % 5 == 0]  
  
# 测试  
print(multiples_of_3_or_5(15))
```

程序逻辑解析:

使用列表推导式筛选满足条件的数字，简洁实现过滤功能。

程序语法要点:

- 列表推导式 `[num for num in range(... ) if condition]`
 - 多重条件 `num % 3 == 0 or num % 5 == 0`
-

示例12: 计算 $1^3+2^3+...+n^3$

```
def sum_of_cubes(n):  
    return sum(i**3 for i in range(1, n+1))  
  
# 测试  
print(sum_of_cubes(3))
```

程序逻辑解析:

使用生成器表达式计算立方和，利用sum函数高效求和。

程序语法要点:

- 生成器表达式 `sum(i**3 for i in range(...))`
 - 立方运算 `i**3`
-

示例13: 判断三位数是否为水仙花数

```
def is_narcissistic_number(num):  
    if not (isinstance(num, int) and 100 <= num <= 999):  
        return False  
    hundreds = num // 100  
    tens = (num // 10) % 10
```

```
units = num % 10
return hundreds**3 + tens**3 + units**3 == num

# 测试
print(is_narcissistic_number(153))
```

程序逻辑解析：

分离三位数的各位数字，验证立方和是否等于原数。

程序语法要点：

- 类型和范围验证 `isinstance(num, int)` and `100 <= num <= 999`
- 数位分离 `hundreds = num // 100`等

示例14：生成n个随机整数

```
import random
def generate_random_integers(min_val, max_val, n):
    return [random.randint(min_val, max_val) for _ in range(n)]

# 测试
print(generate_random_integers(1, 10, 5))
```

程序逻辑解析：

使用列表推导式调用`random.randint`生成指定范围内的随机整数列表。

程序语法要点：

- 随机数生成 `random.randint(min_val, max_val)`
- 列表推导式应用 `[... for _ in range(n)]`

示例15：返回1到n之间的所有素数

```
def primes_up_to(n):
    if n < 2:
        return []
    primes = []
    for num in range(2, n+1):
        is_prime = True
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0:
                is_prime = False
                break
        if is_prime:
            primes.append(num)
    return primes
```

```
# 测试
print(primes_up_to(30))
```

程序逻辑解析：

使用试除法判断每个数字是否为素数，优化到平方根范围检查。

程序语法要点：

- **素数判断优化** `range(2, int(num**0.5) + 1)`
- **循环提前退出** `break`发现因子立即终止

示例16：两数相除返回商和余数

```
def divide_with_remainder(a, b):
    if b == 0:
        raise ValueError("除数不能为 0")
    quotient = a // b
    remainder = a % b
    return (quotient, remainder)
# 测试
print(divide_with_remainder(10, 3))
```

程序逻辑解析：

使用整除和取余运算分别计算商和余数，以元组形式返回。

程序语法要点：

- **元组返回** `return (quotient, remainder)`
- **除零检查** `if b == 0: raise ValueError(...)`

示例17：返回1到n之间的所有偶数

```
def even_numbers_up_to(n):
    return [num for num in range(2, n+1, 2)]
# 测试
print(even_numbers_up_to(10))
```

程序逻辑解析：

利用range函数的步长参数直接生成偶数序列。

程序语法要点：

- **步长参数** `range(2, n+1, 2)`
- **列表构造** 直接由range转换为列表

示例18：接收多个参数返回乘积

```
def multiply_numbers(*args):
    if not args:
        return 0
    product = 1
    for num in args:
        product *= num
    return product
# 测试
print(multiply_numbers(2, 3, 4))
```

使用可变参数接收任意数量参数，遍历计算所有参数的乘积。

程序语法要点：

- 可变参数 `*args`
- 空参数处理 `if not args: return 0`

示例19：判断整数是否为奇数

```
def is_odd(num):
    return num % 2 != 0
# 测试
print(is_odd(5))
print(is_odd(4))
```

程序逻辑解析：

通过取余运算检查数字除以2的余数是否为1来判断奇数。

程序语法要点：

- 布尔直接返回 `return num % 2 != 0`
- 简洁表达式 无需if-else直接返回比较结果

示例20：计算e的近似值（按项数）

```
def calculate_e(n):
    if n < 0:
        raise ValueError("n必须是非负整数")
    e = 1.0
    factorial = 1.0
    for i in range(1, n+1):
        factorial *= i
```

```
    e += 1 / factorial
    return e
```

程序逻辑解析:

通过泰勒级数展开计算自然常数e的近似值，累加前n项。

程序语法要点:

- 阶乘累积 `factorial *= i`
- 级数累加 `e += 1 / factorial`

示例21: 计算e的近似值 (按精度)

```
def calculate_e_precision():
    e = 1.0
    factorial = 1.0
    n = 0
    precision = 1e-6
    while True:
        n += 1
        factorial *= n
        term = 1 / factorial
        e += term
        if term < precision:
            break
    return e, n
```

程序逻辑解析:

根据精度要求动态计算项数，当新增项小于精度阈值时停止计算。

程序语法要点:

- 精度控制循环 `while True:` + `break` 条件
- 动态项数 `n += 1` 递增计算

示例22: 解一元二次方程

```
def calc_equation(a, b, c):
    beta = b*b - 4*a*c
    if beta >= 0:
        x1 = (-b + beta**0.5) / (2*a)
        x2 = (-b - beta**0.5) / (2*a)
        print("x1=%.2f, x2=%.2f" % (x1, x2))
    else:
        print("没有实根")
```

```
# 测试
calc_equation(2, 3.0, 1)
calc_equation(3, 8.0, 2)
calc_equation(2, 2.0, 1)
```

程序逻辑解析：

根据判别式判断实根情况，应用求根公式计算方程解。

程序语法要点：

- 判别式计算 `beta = b*b - 4*a*c`
- 格式化输出 `"x1=%.2f, x2=%.2f" % (x1, x2)`

第九讲 Python程序实践二

示例1：返回字符串中非空格字符总长度

```
def str_length_without_spaces(s):
    return len(s.replace(" ", ""))
# 测试
print(str_length_without_spaces("Hello world"))
```

程序逻辑解析：

通过替换空格字符为空字符串，计算剩余字符串的长度得到非空格字符总数。

程序语法要点：

- 字符串替换 `s.replace(" ", "")`
- 长度计算 `len()` 处理替换后的字符串

示例2：统计元音字母数量

```
def count_vowels(s):
    vowels = ('a', 'e', 'i', 'o', 'u')
    return sum(1 for char in s.lower() if char in vowels)
# 测试
print(count_vowels("Hello World"))
```

程序逻辑解析：

遍历字符串中的每个字符，检查是否为元音字母并计数。

程序语法要点：

- 元音元组 `vowels = ('a', 'e', 'i', 'o', 'u')`

- 生成器表达式求和 `sum(1 for char in ... if condition)`
-

示例3：计算字符串中所有数字总和

```
def sum_numbers_in_string(s):
    total = 0
    for char in s:
        if char.isdigit():
            total += int(char)
    return total
# 测试
print(sum_numbers_in_string("abc123def45"))
```

程序逻辑解析：

遍历字符串中的每个字符，识别数字字符并转换为整数累加。

程序语法要点：

- 数字判断 `char.isdigit()`
 - 类型转换累加 `total += int(char)`
-

示例4：检查字符串是否由纯数字组成

```
def is_all_digits(s):
    return s.isdigit()

# 测试
print(is_all_digits("12345"))
print(is_all_digits("12a34"))
```

程序逻辑解析：

使用字符串方法直接验证字符串是否全部由数字字符组成。

程序语法要点：

- 字符串方法 `s.isdigit()`
 - 直接返回布尔值 无需显式条件判断
-

示例5：返回单词平均长度

```
def average_word_length(s):
    words = s.split()
    if not words:
```

```
        return 0
    total_length = sum(len(word) for word in words)
    return total_length / len(words)

# 测试
print(average_word_length("Hello world this is python"))
```

程序逻辑解析：

分割字符串为单词列表，计算所有单词长度总和除以单词数量。

程序语法要点：

- 空列表检查 `if not words: return 0`
- 生成器表达式求和 `sum(len(word) for word in words)`

示例6：反转每个单词字符顺序

```
def reverse_words(s):
    words = s.split()
    reversed_words = [word[::-1] for word in words]
    return " ".join(reversed_words)

# 测试
print(reverse_words("Hello world"))
```

程序逻辑解析：

分割字符串为单词，对每个单词进行字符反转，然后重新连接。

程序语法要点：

- 字符串反转 `word[::-1]`
- 列表推导式 `[word[::-1] for word in words]`

示例7：每个单词首字母大写

```
def capitalize_first_letter(s):
    words = s.split()
    capitalized = [word[0].upper() + word[1:].lower() for word in words]
    return " ".join(capitalized)

# 测试
print(capitalize_first_letter("hello world PYTHON"))
```

程序逻辑解析：

分割字符串为单词，将每个单词的首字母大写，其余字母小写，然后重新连接。

程序语法要点：

- 切片操作 `word[1:].lower()`
 - 字符串连接 `" ".join(capitalized)`
-

示例8：统计大写字母个数

```
def count_uppercase_letters(s):  
    return sum(1 for char in s if char.isupper())  
# 测试  
print(count_uppercase_letters("Hello World PYTHON"))
```

程序逻辑解析：

遍历字符串中的每个字符，检查是否为大写字母并计数。

程序语法要点：

- 大写判断 `char.isupper()`
 - 生成器表达式 `sum(1 for char in s if condition)`
-

示例9：判断字符串是否只包含字母

```
def is_all_letters(s):  
    return s.isalpha()  
# 测试  
print(is_all_letters("HelloWorld"))  
print(is_all_letters("Hello123"))
```

程序逻辑解析：

使用字符串方法验证字符串是否全部由字母字符组成。

程序语法要点：

- 字母验证 `s.isalpha()`
 - 直接返回方法结果 简洁高效
-

示例10：替换空格为下划线

```
def replace_spaces_with_underscores(s):  
    return s.replace(" ", "_")  
# 测试  
print(replace_spaces_with_underscores("Hello world"))
```

程序逻辑解析：

使用字符串替换方法将所有空格字符替换为下划线。

程序语法要点：

- 简单替换 `s.replace(" ", "_")`
 - 直接返回结果 无需中间变量
-

示例11：两个字符出现次数差

```
def char_count_difference(s, char1, char2):  
    count1 = s.count(char1)  
    count2 = s.count(char2)  
    return count1 - count2  
# 测试  
print(char_count_difference("abracadabra", "a", "b"))
```

程序逻辑解析：

分别统计两个字符在字符串中的出现次数，计算它们的差值。

程序语法要点：

- 字符计数 `s.count(char1)`
-

示例12：转换为全小写字符串

```
def to_lowercase_string(s):  
    return s.lower()  
# 测试  
print(to_lowercase_string("Hello WORLD"))
```

程序逻辑解析：

使用字符串方法将字符串中的所有字符转换为小写形式。

程序语法要点：

- 大小写转换 `s.lower()`
-

示例13：子串最后一次出现索引

```
def last_occurrence(s, substr):  
    substr_len = len(substr)  
    s_len = len(s)  
    if substr_len == 0 or substr_len > s_len:
```

```

        return -1
    for i in range(s_len - substr_len, -1, -1):
        if s[i:i + substr_len] == substr:
            return i
    return -1
# 测试
print(last_occurrence("abracadabra", "abr"))

```

程序逻辑解析：

从字符串末尾向前搜索，找到目标子串最后一次出现的位置。

程序语法要点：

- 反向遍历 `range(s_len - substr_len, -1, -1)`
- 子串比较 `s[i:i + substr_len] == substr`

示例14：每个字符重复n次

```

def repeat_chars(s, n):
    return "".join([char * n for char in s])
# 测试
print(repeat_chars("abc", 2))

```

程序逻辑解析：

遍历字符串中的每个字符，将其重复指定次数后连接成新字符串。

程序语法要点：

- 字符重复 `char * n`
- 列表推导式连接 `"".join([char * n for char in s])`

示例15：判断是否以指定后缀结尾

```

def ends_with_suffix(s, suffix):
    suffix_len = len(suffix)
    s_len = len(s)
    if suffix_len > s_len:
        return False
    return s[-suffix_len:] == suffix
# 测试
print(ends_with_suffix("Hello World", "World"))

```

程序逻辑解析：

比较字符串末尾部分是否与指定后缀相同。

程序语法要点:

- 切片比较 `s[-suffix_len:] == suffix`
 - 长度检查 `if suffix_len > s_len: return False`
-

示例16: 按长度排序字符串列表

```
def sort_strings_by_length(strings):  
    return sorted(strings, key=lambda x: (len(x), x))  
# 测试  
print(sort_strings_by_length(["apple", "banana", "pear", "kiwi"]))
```

程序逻辑解析:

使用排序函数，按字符串长度和字母顺序对字符串列表进行排序。

程序语法要点:

- 多键排序 `key=lambda x: (len(x), x)`
 - `sorted`函数 返回新列表
-

示例17: 统计小写字母个数

```
def count_lowercase_letters(s):  
    return sum(1 for char in s if char.islower())  
# 测试  
print(count_lowercase_letters("Hello world Python"))
```

程序逻辑解析:

遍历字符串中的每个字符，检查是否为小写字母并计数。

程序语法要点:

- 小写判断 `char.islower()`
 - 生成器表达式求和 简洁计数
-

示例18: 判断是否包含数字

```
def contains_digit(s):  
    for char in s:  
        if char.isdigit():  
            return True  
    return False  
# 测试
```

```
print(contains_digit("Hello123"))
print(contains_digit("Hello"))
```

程序逻辑解析：

遍历字符串中的每个字符，发现数字字符立即返回True。

程序语法要点：

- **提前返回** 发现数字立即返回True
- **循环遍历检查** 逐个字符判断

示例19：替换下划线为空格

```
def replace_underscores_with_spaces(s):
    return s.replace("_", " ")
# 测试
print(replace_underscores_with_spaces("Hello_world"))
```

程序逻辑解析：

使用字符串替换方法将所有下划线字符替换为空格。

程序语法要点：

- **简单替换** `s.replace("_", " ")`
- **直接返回结果** 方法链式调用

示例20：统计数字字符数量

```
def count_digits(s):
    return sum(1 for char in s if char.isdigit())
# 测试
print(count_digits("abc123def456"))
```

程序逻辑解析：

遍历字符串中的每个字符，识别数字字符并计数。

程序语法要点：

- **数字判断** `char.isdigit()`
- **生成器表达式** `sum(1 for char in s if condition)`

示例21：判断字符串是否为空

```
def is_string_empty(s):  
    return len(s) == 0  
# 测试  
print(is_string_empty("hello"))  
print(is_string_empty(""))
```

程序逻辑解析：

检查字符串长度是否为0来判断是否为空字符串。

程序语法要点：

- 长度检查 `len(s) == 0`
 - 直接返回比较结果
-

示例22：转换为全大写字符串

```
def to_uppercase_string(s):  
    return s.upper()  
# 测试  
print(to_uppercase_string("Hello World"))
```

程序逻辑解析：

使用字符串方法将字符串中的所有字符转换为大写形式。

程序语法要点：

- 大小写转换 `s.upper()`
 - 直接返回转换结果
-

示例23：子串第一次出现索引

```
def first_occurrence(s, substr):  
    substr_len = len(substr)  
    s_len = len(s)  
    if substr_len == 0 or substr_len > s_len:  
        return -1  
    for i in range(s_len - substr_len + 1):  
        if s[i:i + substr_len] == substr:  
            return i  
    return -1  
# 测试  
print(first_occurrence("abracadabra", "abr"))
```

程序逻辑解析：

从字符串开头向后搜索，找到目标子串第一次出现的位置。

程序语法要点：

- **正向遍历** `range(s_len - substr_len + 1)`
 - **子串匹配** `s[i:i + substr_len] == substr`
-

示例24：统计非字母字符个数

```
def count_non_letters(s):  
    return sum(1 for char in s if not char.isalpha())  
# 测试  
print(count_non_letters("Hello, World! 123"))
```

程序逻辑解析：

遍历字符串中的每个字符，检查是否为非字母字符并计数。

程序语法要点：

- **非字母判断** `not char.isalpha()`
 - **生成器表达式求和** 简洁实现
-

示例25：判断是否包含字母

```
def contains_letter(s):  
    for char in s:  
        if char.isalpha():  
            return True  
    return False  
# 测试  
print(contains_letter("123ab456"))  
print(contains_letter("123456"))
```

程序逻辑解析：

遍历字符串中的每个字符，发现字母字符立即返回True。

程序语法要点：

- **字母判断** `char.isalpha()`
 - **提前返回优化** 发现字母立即结束
-

示例26：字符串首字母大写

```
def capitalize_first(s):
    if not s:
        return s
    return s[0].upper() + s[1:].lower()
# 测试
print(capitalize_first("hello world"))
```

程序逻辑解析：

将字符串的第一个字符大写，其余字符转换为小写。

程序语法要点：

- 空字符串检查 `if not s: return s`
- 切片操作 `s[1:].lower()`

示例27：统计字母字符数量

```
def count_letters(s):
    return sum(1 for char in s if char.isalpha())
# 测试
print(count_letters("Hello, World! 123"))
```

程序逻辑解析：

遍历字符串中的每个字符，识别字母字符并计数。

程序语法要点：

- 字母判断 `char.isalpha()`
- 生成器表达式 `sum(1 for char in s if condition)`

示例28：凯撒密码

```
def shift_char(c):
    if c.islower():
        return chr((ord(c) - ord('a') + 3) % 26 + ord('a'))
    elif c.isupper():
        return chr((ord(c) - ord('A') + 3) % 26 + ord('A'))
    else:
        return c
def shift_text(text):
    return "".join(shift_char(c) for c in text)
# 测试
print(shift_text("ABCDEFGHIIJKLMNOPQRSTUVWXYZ"))
```

程序逻辑解析：

对每个字母字符进行移位加密，非字母字符保持不变。

程序语法要点：

- 字符编码操作 `ord(c)` 和 `chr()`
 - 模运算 `(ord(c) - ord('a') + 3) % 26`
-

示例29：中英文分词

```
def is_chinese(uc):
    ac = ord(uc)
    if ac >= 0x4E00 and ac <= 0x9FA5:
        return True
    else:
        return False

def seg_word():
    s = input("input sentence:")
    s1, s2 = "", ""
    switch = 1 # 1表示当前可加中文，0表示当前可加英文
    for uc in s:
        if is_chinese(uc):
            if switch == 0:
                switch = 1
            s1 += uc
        else:
            if switch == 1:
                switch = 0
            s2 += uc
    print("中文:%s。" % s1, "英文:%s。" % s2)
```

程序逻辑解析：

根据Unicode编码范围区分中英文字符，分别收集到不同字符串中。

程序语法要点：

- Unicode范围判断 `ac >= 0x4E00 and ac <= 0x9FA5`
 - 状态切换 `switch` 变量控制当前收集类型
-

示例30：表达式解析与计算

```
def calculate_expression(expr):
    # 辅助函数：计算两个整数的运算结果
    def calc(a, op, b):
        a, b = int(a), int(b)
```

```

    if op == '+':
        return a + b
    elif op == '-':
        return a - b
    elif op == '*':
        return a * b
    elif op == '/':
        if b == 0:
            raise ValueError("除数不能为0")
        return a // b # 整数除法
    else:
        raise ValueError(f"不支持的运算符:{op}")

# 辅助函数：解析不带括号的表达式
def parse_single(expr_part):
    for op in '+-*/':
        if op in expr_part:
            left, right = expr_part.split(op, 1)
            return calc(left, op, right), left, op, right
    raise ValueError("无效的子表达式")

# 处理括号
def process_parentheses(expr):
    steps = []
    step_num = 1
    while '(' in expr:
        # 找到最右侧的左括号
        left_idx = expr.rfind('(')
        # 找到对应的右括号
        right_idx = expr.find(')', left_idx)
        # 提取子表达式
        sub_expr = expr[left_idx + 1:right_idx]
        # 计算子表达式
        result, num1, op, num2 = parse_single(sub_expr)
        # 记录步骤
        steps.append(f"第{step_num}步计算 v{step_num}={num1}{op}{num2}={result}")
        step_num += 1
        # 替换子表达式为结果
        expr = expr[:left_idx] + str(result) + expr[right_idx + 1:]
    # 处理最后的表达式
    if expr:
        result, num1, op, num2 = parse_single(expr)
        steps.append(f"第{step_num}步计算 v{step_num}={num1}{op}{num2}={result}")
    return steps, result

try:
    steps, final_result = process_parentheses(expr)
    for step in steps:
        print(step)
    print(f"最终结果: {final_result}")

```

```
        return final_result
    except Exception as e:
        print(f"错误:{e}")
        return None

# 测试
if __name__ == '__main__':
    print("测试1:(3+8)*(2+4)")
    calculate_expression("(3+8)*(2+4)")
    print("\n测试2:((5-2)+(10/2))*(3+7)")
    calculate_expression("((5-2)+(10/2))*(3+7)")
    print("\n测试3:(100-(20+30))*(5*2)")
    calculate_expression("(100-(20+30))*(5*2)")
```

程序逻辑解析:

递归处理括号表达式，从内到外逐步计算并记录计算步骤。

程序语法要点:

- 递归解析 `process_parentheses` 处理嵌套括号
- 步骤记录 `steps.append(f"第{step_num}步计算...")`
- 异常处理 `try-except` 捕获计算错误

第10讲 Python程序实践三

示例1：计算列表中所有偶数的总和

```
def sum_even_numbers(lst):
    return sum(num for num in lst if num % 2 == 0)

# 测试
print(sum_even_numbers([1,2,3,4,5,6]))# 输出:12
```

程序逻辑解析:

使用列表推导式筛选出所有偶数，然后通过sum函数计算这些偶数的总和。

程序语法要点:

- 条件列表推导式 `[num for num in lst if num % 2 == 0]`
- sum函数应用 直接对生成器表达式求和

示例2：判断列表是否按升序排序

```
def is_sorted(lst):
    for i in range(len(lst)-1):
```



```
        if lst[i] > lst[i+1]:
            return False
    return True
# 测试
print(is_sorted([1,2,3,4]))# 输出:True
print(is_sorted([1,3,2,4]))# 输出:False
```

程序逻辑解析:

遍历列表中的相邻元素对，检查是否每个元素都不大于其后继元素。

程序语法要点:

- **相邻元素比较** `lst[i] > lst[i+1]`
- **提前返回** 发现逆序立即返回False

示例3: 返回列表中所有元素的平均值(忽略非数字元素)

```
def average(lst):
    numbers = [x for x in lst if isinstance(x,(int, float))]
    if not numbers:
        return 0
    return sum(numbers)/len(numbers)
# 测试
print(average([1,2,3,'a',4]))# 输出:2.5
```

程序逻辑解析:

过滤出数字元素，计算总和后除以元素数量得到平均值。

程序语法要点:

- **类型检查** `isinstance(x, (int, float))`
- **空列表处理** `if not numbers: return 0`

示例4: 返回列表中第二大的元素(假设列表至少有两个不同元素)

```
def second_largest(lst):
    unique = list(set(lst))
    unique.sort()
    return unique[-2]
# 测试
print(second_largest([5,3,8,8,2]))# 输出:5
```

程序逻辑解析:

通过集合去重后排序，返回倒数第二个元素作为第二大的值。

程序语法要点：

- **集合去重** `list(set(lst))`
 - **排序索引** `unique[-2]` 获取第二大元素
-

示例5：返回一个新列表，其中每个元素是原列表对应元素的2倍

```
def double_elements(lst):  
    return [x*2 for x in lst]  
# 测试  
print(double_elements([1,2,3,4]))# 输出:[2,4,6,8]
```

程序逻辑解析：

使用列表推导式对原列表中的每个元素进行乘以2的操作。

程序语法要点：

- **元素变换** `[x*2 for x in lst]`
 - **列表推导式** 简洁的元素处理
-

示例6：返回存在于第一个列表但不存在于第二个列表的元素

```
def list_difference(list1, list2):  
    return [x for x in list1 if x not in list2]  
# 测试  
print(list_difference([1,2,3,4], [3,4,5]))# 输出:[1,2]
```

程序逻辑解析：

找出第一个列表中存在但第二个列表中不存在的所有元素。

程序语法要点：

- **成员检查** `x not in list2`
 - **条件列表推导式** 过滤满足条件的元素
-

示例7：移除列表重复元素后，将列表逆序排列

```
def unique_reversed(lst):  
    unique = []  
    seen = set()  
    for item in lst:  
        if item not in seen:  
            seen.add(item)  
            unique.append(item)
```

```
    return unique[::-1]

# 测试
print(unique_reversed([1,2,2,3,1]))# 输出:[3,2,1]
```

程序逻辑解析:

使用集合跟踪已见元素，保持首次出现顺序去重，然后反转列表。

程序语法要点:

- 集合跟踪 `seen = set()` 记录已出现元素
- 切片反转 `unique[::-1]` 逆序排列

示例8: 返回所有小于等于n的斐波那契数列表

```
def fibonacci_upto(n):
    fibs = []
    a, b = 0, 1
    while a <= n:
        fibs.append(a)
        a, b = b, a + b
    return fibs

# 测试
print(fibonacci_upto(10))# 输出:[0,1,1,2,3,5,8]
```

程序逻辑解析:

使用迭代方法生成斐波那契数列，直到当前值超过n为止。

程序语法要点:

- 变量交换 `a, b = b, a + b`
- 循环条件 `while a <= n`

示例9: 返回一个元组，包含正数和负数的数量(0既不算正数也不算负数)

```
def count_positives_negatives(lst):
    positives = 0
    negatives = 0
    for num in lst:
        if num > 0:
            positives += 1
        elif num < 0:
            negatives += 1
    return (positives, negatives)
```

```
# 测试
print(count_positives_negatives([1,-2,3,0,-5,6]))# 输出:(3,2)
```

程序逻辑解析:

遍历列表分类统计正数和负数的数量，忽略零值。

程序语法要点:

- **分类计数** 分别对正数和负数计数
- **元组返回** `return (positives, negatives)`

示例10: 返回列表中出现次数最少的元素(可能有多)

```
def least_frequent_elements(lst):
    if not lst:
        return []
    counts = {}
    for item in lst:
        counts[item] = counts.get(item, 0) + 1
    min_count = min(counts.values())
    return [item for item, count in counts.items() if count == min_count]
# 测试
print(least_frequent_elements([1,2,2,3,3,3,4]))# 输出:[1,4]
```

程序逻辑解析:

使用字典统计每个元素的频率，找到最小频率对应的所有元素。

程序语法要点:

- **频率统计** `counts.get(item, 0) + 1`
- **最小值筛选** `[item for item, count in counts.items() if count == min_count]`

示例11: 返回列表中所有偶数组成的新列表

```
def filter_even_numbers(lst):
    return [num for num in lst if num % 2 == 0]
# 测试
print(filter_even_numbers([1,2,3,4,5,6]))# 输出:[2,4,6]
```

程序逻辑解析:

使用列表推导式筛选出所有能被2整除的元素。

程序语法要点:

- **偶数判断** `num % 2 == 0`

- **列表推导式过滤** 简洁的条件筛选
-

示例12：返回列表中所有数字元素的乘积(忽略非数字元素)

```
def product_of_elements(lst):
    product = 1
    has_numbers = False
    for item in lst:
        if isinstance(item, (int, float)):
            product *= item
            has_numbers = True
    return product if has_numbers else 0
# 测试
print(product_of_elements([1,2,3,'a',4]))# 输出:24
```

程序逻辑解析：

遍历列表，只对数字元素进行乘法运算，忽略非数字元素。

程序语法要点：

- **类型检查** `isinstance(item, (int, float))`
 - **乘积累积** `product *= item`
-

示例13：检查列表中是否有重复的元素

```
def has_duplicates(lst):
    return len(lst) != len(set(lst))
# 测试
print(has_duplicates([1,2,3,4]))# 输出:False
print(has_duplicates([1,2,2,3]))# 输出:True
```

程序逻辑解析：

遍历列表，只对数字元素进行乘法运算，忽略非数字元素。

程序语法要点：

- **类型检查** `isinstance(item, (int, float))`
 - **乘积累积** `product *= item`
-

示例14：接收多个数字参数，返回它们的总和

```
def add_numbers(*args):
    return sum(args)
```

```
# 测试
print(add_numbers(1,2,3,4))# 输出:10
```

程序逻辑解析:

使用可变参数接收任意数量的数字，计算它们的总和。

程序语法要点:

- 可变参数 `*args`
- `sum`函数 直接对参数元组求和

示例15: 返回一个新列表，所有元素都转换为字符串类型

```
def convert_to_strings(lst):
    return [str(item) for item in lst]
# 测试
print(convert_to_strings([1,2.5,True,"hello"]))# 输出:['1','2.5','True','hello']
```

程序逻辑解析:

使用列表推导式对列表中的每个元素应用`str`函数进行类型转换。

程序语法要点:

- 类型转换 `str(item)`
- 列表推导式 简洁的元素转换

示例16: 将多个列表合并成一个新列表

```
def merge_lists(lists):
    merged = []
    for lst in lists:
        merged.extend(lst)
    return merged
# 测试
print(merge_lists([[1,2],[3,4],[5,6]]))# 输出:[1,2,3,4,5,6]
```

程序逻辑解析:

遍历所有子列表，使用`extend`方法将它们合并到一个新列表中。

程序语法要点:

- 列表扩展 `merged.extend(lst)`
- 循环合并 逐个处理子列表

示例17：返回列表中最小的n个元素(不改变顺序)

```
def smallest_n_elements(lst, n):
    if n <= 0:
        return []
    indexed = [(val, i) for i, val in enumerate(lst)]
    indexed.sort()
    smallest_indices = {i for val, i in indexed[:n]}
    return [val for i, val in enumerate(lst) if i in smallest_indices]
# 测试
print(smallest_n_elements([5,3,8,1,2],3))# 输出:[3,1,2]
```

程序逻辑解析：

通过索引跟踪原始位置，排序后按原始顺序返回最小的n个元素。

程序语法要点：

- 索引跟踪 [(val, i) for i, val in enumerate(lst)]
- 集合成员检查 if i in smallest_indices

示例18：判断一个列表是否为空列表

```
def is_list_empty(lst):
    return len(lst) == 0
# 测试
print(is_list_empty([]))# 输出:True
print(is_list_empty([1,2]))# 输出:False
```

程序逻辑解析：

检查列表长度是否为0来判断列表是否为空。

程序语法要点：

- 长度检查 len(lst) == 0
- 直接返回比较结果

示例19：返回列表中所有数字元素的总和(忽略非数字元素)

```
def sum_elements(lst):
    return sum(x for x in lst if isinstance(x,(int, float)))
# 测试
print(sum_elements([1,2,'a',3.5]))# 输出:6.5
```

程序逻辑解析：

使用生成器表达式筛选数字元素并计算它们的总和。

程序语法要点：

- **类型过滤** `isinstance(x, (int, float))`
 - **生成器表达式** `sum(x for x in lst if condition)`
-

示例20：不使用[::-1]语法，返回反转后的列表

```
def reverse_list(lst):
    reversed_lst = []
    for i in range(len(lst)-1, -1, -1):
        reversed_lst.append(lst[i])
    return reversed_lst
# 测试
print(reverse_list([1,2,3,4]))# 输出:[4,3,2,1]
```

程序逻辑解析：

从列表末尾向前遍历，将元素逐个添加到新列表中实现反转。

程序语法要点：

- **反向遍历** `range(len(lst)-1, -1, -1)`
 - **手动构建** `reversed_lst.append(lst[i])`
-

示例21：判断两个列表的元素和顺序是否完全相同

```
def are_lists_equal(lst1, lst2):
    if len(lst1) != len(lst2):
        return False
    for a, b in zip(lst1, lst2):
        if a != b:
            return False
    return True
# 测试
print(are_lists_equal([1,2,3], [1,2,3]))# 输出:True
print(are_lists_equal([1,2,3], [1,3,2]))# 输出:False
```

程序逻辑解析：

先比较长度，再逐个比较对应位置的元素是否相等。

程序语法要点：

- **长度检查** `if len(lst1) != len(lst2)`
- **zip函数** `for a, b in zip(lst1, lst2)`

示例22：返回列表中最大的数字元素(忽略非数字元素)

```
def max_number(lst):
    numbers = [x for x in lst if isinstance(x, (int, float))]
    if not numbers:
        return None
    return max(numbers)
# 测试
print(max_number([1, 5, 'a', 3.5, 10]))# 输出:10
```

程序逻辑解析：

过滤出数字元素后，使用max函数找到其中的最大值。

程序语法要点：

- **数字过滤** `[x for x in lst if isinstance(x, (int, float))]`
- **max函数应用** 对数字列表求最大值

示例23：按字符串长度从短到长排序，长度相同则按字母顺序

```
def sort_strings_by_length(strings):
    return sorted(strings, key=lambda x: (len(x), x))
# 测试
print(sort_strings_by_length(["apple", "banana", "pear", "kiwi"]))# 输出:
['kiwi', 'pear', 'apple', 'banana']
```

程序逻辑解析：

使用多键排序，先按长度排序，长度相同的按字母顺序排序。

程序语法要点：

- **多键排序** `key=lambda x: (len(x), x)`
- **sorted函数** 返回排序后的新列表

示例24：返回一个新列表，所有可转换为整数的元素都转换为整数，否则保持原样

```
def convert_to_integers(lst):
    result = []
    for item in lst:
        try:
            result.append(int(item))
        except (ValueError, TypeError):
            result.append(item)
    return result
```

```
# 测试
print(convert_to_integers(["1", "2.5", "3", "four", 5.0]))# 输出:[1, '2.5', 3, 'four', 5]
```

程序逻辑解析:

尝试将每个元素转换为整数，转换失败时保持原样。

程序语法要点:

- 异常处理 `try-except` 捕获转换错误
- 类型转换尝试 `int(item)`

示例25: 返回列表中最大的n个元素(不改变顺序)

```
def largest_n_elements(lst, n):
    if n <= 0:
        return []
    indexed = [(val, i) for i, val in enumerate(lst)]
    indexed.sort(reverse=True, key=lambda x:x[0])
    largest_indices = {i for val, i in indexed[:n]}
    return [val for i, val in enumerate(lst) if i in largest_indices]
# 测试
print(largest_n_elements([5,3,8,1,2],3))# 输出:[5,8,2]
```

程序逻辑解析:

通过索引跟踪原始位置，降序排序后按原始顺序返回最大的n个元素。

程序语法要点:

- 降序排序 `sort(reverse=True, key=lambda x:x[0])`
- 索引集合 `{i for val, i in indexed[:n]}`

示例26: 返回列表中最小的数字元素(忽略非数字元素)

```
def min_number(lst):
    numbers = [x for x in lst if isinstance(x, (int, float))]
    if not numbers:
        return None
    return min(numbers)
# 测试
print(min_number([1,5,'a',3.5,0]))# 输出:0
```

程序逻辑解析:

过滤出数字元素后，使用min函数找到其中的最小值。

程序语法要点:

- **数字过滤** 同示例22
 - **min函数应用** 对数字列表求最小值
-

示例27：返回一个新列表，所有可转换为浮点数的元素都转换为浮点数，否则保持原样

```
def convert_to_floats(lst):
    result = []
    for item in lst:
        try:
            result.append(float(item))
        except (ValueError, TypeError):
            result.append(item)
    return result
# 测试
print(convert_to_floats(["1", "2.5", "three", 4]))# 输出:[1.0, 2.5, 'three', 4.0]
```

程序逻辑解析：

尝试将每个元素转换为浮点数，转换失败时保持原样。

程序语法要点：

- **浮点数转换** `float(item)`
 - **异常处理** 同示例24
-

示例28：返回元素在列表中第一次出现的索引，没有则返回-1

```
def find_index(lst, item):
    for i, value in enumerate(lst):
        if value == item:
            return i
    return -1
# 测试
print(find_index([1, 2, 3, 2, 1], 2))# 输出:1
```

程序逻辑解析：

遍历列表，找到目标元素第一次出现的位置。

程序语法要点：

- **enumerate函数** `for i, value in enumerate(lst)`
 - **提前返回** 找到目标立即返回索引
-

示例29：返回包含 $1^2, 2^2, \dots, n^2$ 的列表

```
def square_numbers(n):  
    return [i*i for i in range(1, n+1)]  
# 测试  
print(square_numbers(5))# 输出:[1,4,9,16,25]
```

程序逻辑解析:

使用列表推导式生成从1到n的平方数序列。

程序语法要点:

- 平方计算 `i*i`
- 范围生成 `range(1, n+1)`

示例30: 生成从start到end(包含), 步长为step的等差数列

```
def arithmetic_sequence(start, end, step):  
    if step == 0:  
        raise ValueError("步长不能为 0")  
    sequence = []  
    current = start  
    if step > 0:  
        while current <= end:  
            sequence.append(current)  
            current += step  
    else:  
        while current >= end:  
            sequence.append(current)  
            current += step  
    return sequence  
# 测试  
print(arithmetic_sequence(1,10,2))# 输出:[1,3,5,7,9]
```

程序逻辑解析:

根据起始值、终止值和步长生成等差数列, 处理正负步长情况。

程序语法要点:

- 步长检查 `if step == 0: raise ValueError`
- 方向判断 `if step > 0:` 和 `else` 分支

示例31: 将一个二维列表(矩阵)转置

```
def transpose_matrix(matrix):  
    if not matrix or not matrix[0]:  
        return []  
    rows = len(matrix)
```

```

    cols = len(matrix[0])
    transposed = []
    for j in range(cols):
        new_row = []
        for i in range(rows):
            new_row.append(matrix[i][j])
        transposed.append(new_row)
    return transposed
# 测试
matrix = [[1,2,3],[4,5,6],[7,8,9]]
print(transpose_matrix(matrix))# 输出:[[1,4,7],[2,5,8],[3,6,9]]

```

程序逻辑解析：

将矩阵的行列互换，构建新的转置矩阵。

程序语法要点：

- 行列索引交换 `matrix[i][j] → transposed[j][i]`
- 嵌套循环 外层列循环，内层行循环

示例32：计算一个三维列表中所有元素的总和

```

def sum_3d_list(lst):

    total = 0
    for layer in lst:
        for row in layer:
            for num in row:
                total += num
    return total

# 测试
three_d_list = [[[1,2],[3,4]],[[5,6],[7,8]],[[9,10],[11,12]]]
print(sum_3d_list(three_d_list))# 输出:78

```

程序逻辑解析：

使用三层嵌套循环遍历三维列表的所有元素并累加。

程序语法要点：

- 三层嵌套 `for layer in lst: for row in layer: for num in row`
- 累加求和 `total += num`

示例33：找出二维列表中的最大值及其位置

```
def find_max_in_2d(lst):
    if not lst or not lst[0]:
        return None
    max_val = lst[0][0]
    max_row, max_col = 0, 0
    for i in range(len(lst)):
        for j in range(len(lst[i])):
            if lst[i][j] > max_val:
                max_val = lst[i][j]
                max_row, max_col = i, j
    return (max_val, max_row, max_col)

# 测试
matrix = [[5,2,8],[1,9,3],[4,7,6]]
print(find_max_in_2d(matrix))# 输出:(9,1,1)
```

程序逻辑解析:

遍历二维列表的所有元素，记录最大值及其行列位置。

程序语法要点:

- **行列遍历** 双重循环遍历矩阵
- **位置跟踪** `max_row, max_col = i, j`

示例34：将二维列表转换为一维列表(扁平化)

```
def flatten_2d_list(lst):
    flattened = []
    for row in lst:
        flattened.extend(row)
    return flattened

# 测试
two_d_list = [[1,2,3],[4,5],[6,7,8,9]]
print(flatten_2d_list(two_d_list))# 输出:[1,2,3,4,5,6,7,8,9]
```

程序逻辑解析:

遍历二维列表的每一行，使用extend方法将所有元素合并到一维列表。

程序语法要点:

- **列表扩展** `flattened.extend(row)`
- **简单合并** 无需递归的扁平化

示例35：将二维列表中所有小于0的元素替换为0，大于100的元素替换为100(区间截断)

```
def clamp_2d_list(lst):
    clamped = []
    for row in lst:
        new_row = []

        for num in row:
            if num < 0:
                new_row.append(0)
            elif num > 100:
                new_row.append(100)
            else:
                new_row.append(num)
        clamped.append(new_row)
    return clamped

# 测试
matrix = [[-5, 20, 105], [30, -10, 80], [110, 50, -3]]
print(clamp_2d_list(matrix))# 输出: [[0, 20, 100], [30, 0, 80], [100, 50, 0]]
```

程序逻辑解析:

遍历二维列表，将超出[0,100]范围的数值截断到边界值。

程序语法要点:

- **范围判断** `if num < 0:` 和 `elif num > 100:`
- **新列表构建** 逐元素处理构建新矩阵

第11讲 Python程序实践四

示例1: 接收两个列表，返回它们的交集

```
def find_intersection(list1, list2):
    set1 = set(list1)
    set2 = set(list2)
    return list(set1 & set2)

# 测试
print(find_intersection([1, 2, 3, 4], [3, 4, 5, 6]))# 输出: [3, 4]
```

程序逻辑解析:

将列表转换为集合，使用集合的交集运算找出共同元素，再转换回列表。

程序语法要点:

- **集合交集** `set1 & set2`
- **类型转换** `list(set1 & set2)`

示例2：接收一个列表，返回去重后的列表，保持元素首次出现的顺序

```
def remove_duplicates_preserve_order(lst):
    seen = set()
    result = []
    for item in lst:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result

# 测试
print(remove_duplicates_preserve_order([1,2,2,3,1,4]))# 输出:[1,2,3,4]
```

程序逻辑解析：

使用集合跟踪已见元素，按原始顺序收集首次出现的元素。

程序语法要点：

- **集合成员检查** `if item not in seen`
- **顺序保持** 按遍历顺序添加元素

示例3：判断两个字符串是否是字母异位词

```
def is_anagram(str1, str2):
    # 忽略大小写和空格
    str1_clean = str1.lower().replace(" ", "")
    str2_clean = str2.lower().replace(" ", "")
    return set(str1_clean) == set(str2_clean) and len(str1_clean) == len(str2_clean)

# 测试
print(is_anagram("listen", "silent"))# 输出:True
print(is_anagram("hello", "world"))# 输出:False
```

程序逻辑解析：

清理字符串后比较字符集合和长度，确认是否由相同字母组成。

程序语法要点：

- **字符串清理** `str1.lower().replace(" ", "")`
- **集合和长度比较** `set(str1_clean) == set(str2_clean) and len(str1_clean) == len(str2_clean)`

示例4：接收一个列表，返回列表中只出现一次的元素


```
def find_unique_elements(lst):
    seen = set()
    unique = set()
    for item in lst:
        if item in seen:
            unique.discard(item)
        else:
            seen.add(item)
            unique.add(item)
    return list(unique)
# 测试
print(find_unique_elements([1,2,2,3,3,4]))# 输出:[1,4]
```

程序逻辑解析:

使用两个集合分别跟踪所有元素和唯一元素，发现重复时从唯一集合中移除。

程序语法要点:

- 双集合跟踪 `seen` 和 `unique` 集合
- 集合操作 `unique.discard(item)` 安全移除元素

示例5: 接收两个集合，返回它们的对称差集

```
def symmetric_difference(set1, set2):
    return set1 ^ set2
# 测试
print(symmetric_difference({1,2,3},{3,4,5}))# 输出:{1,2,4,5}
```

程序逻辑解析:

使用集合的对称差集运算符找出只属于一个集合的元素。

程序语法要点:

- 对称差集运算符 `set1 ^ set2`

示例6: 接收一个列表的列表，返回所有子列表的交集

```
def intersection_of_lists(lists):
    if not lists:
        return []
    result = set(lists[0])
    for lst in lists[1:]:
        result.intersection_update(lst)
    if not result:
        break
```

```
    return list(result)
# 测试
print(intersection_of_lists([[1,2,3],[2,3,4],[2,5,6]]))# 输出:[2]
```

程序逻辑解析:

从第一个列表开始，逐步与后续列表求交集，提前终止优化。

程序语法要点:

- 交集更新 `result.intersection_update(lst)`
- 提前终止 `if not result: break`

示例7: 判断一个集合是否是另一个集合的子集

```
def is_subset(subset_candidate, superset_candidate):
    return subset_candidate.issubset(superset_candidate)
# 测试
print(is_subset({1,2},{1,2,3,4}))# 输出:True
print(is_subset({1,5},{1,2,3,4}))# 输出:False
```

程序逻辑解析:

使用集合方法检查一个集合是否是另一个集合的子集。

程序语法要点:

- 子集检查 `subset_candidate.issubset(superset_candidate)`
- 直接返回方法结果

示例8: 接收一个字符串，返回字符串中出现的所有独特字符

```
def unique_chars(s):
    return set(s)
# 测试
print(unique_chars("hello world"))# 输出: {'h','e','l','o',' ','w','r','d'}
```

程序逻辑解析:

将字符串转换为集合，自动去除重复字符。

程序语法要点:

- 集合转换 `set(s)`
- 自动去重 集合特性

示例9: 接收两个列表，返回第一个列表中有而第二个列表中没有的元素

```
def elements_in_first_not_in_second(list1, list2):  
    return list(set(list1) - set(list2))  
# 测试  
print(elements_in_first_not_in_second([1,2,3,4],[3,4,5,6]))# 输出:[1,2]
```

程序逻辑解析:

使用集合差集运算找出第一个集合中独有的元素。

程序语法要点:

- 集合差集 `set(list1) - set(list2)`
- 列表转换 返回列表形式

示例10: 接收一个列表, 返回列表中元素的频率字典

```
def element_frequency(lst):  
    frequency = {}  
    for item in lst:  
        if item in frequency:  
            frequency[item] += 1  
        else:  
            frequency[item] = 1  
    return frequency  
# 测试  
print(element_frequency([1,2,2,3,3,3,4]))# 输出:{1:1,2:2,3:3,4:1}
```

程序逻辑解析:

遍历列表, 使用字典统计每个元素出现的次数。

程序语法要点:

- 字典计数 `frequency[item] = frequency.get(item, 0) + 1`
- `get`方法 处理键不存在的情况

示例11: 接收一个整数n, 返回小于n的所有质数组成的集合

```
def prime_set(n):  
    if n <= 2:  
        return set()  
    primes = set(range(2, n))  
    for i in range(2, int(n**0.5) + 1):  
        if i in primes:  
            primes -= set(range(i*2, n, i))  
    return primes
```

```
# 测试
print(prime_set(20))# 输出:{2,3,5,7,11,13,17,19}
```

程序逻辑解析:

使用埃拉托斯特尼筛法高效找出指定范围内的所有质数。

程序语法要点:

- 筛法实现 `primes -= set(range(i*2, n, i))`
- 集合差集优化 批量移除倍数

示例12: 找出两个字符串集合中长度大于3且同时存在的字符串, 并按字典序排序

```
def common_long_strings(set1, set2):
    common = set1 & set2
    result = [s for s in common if len(s) > 3]
    return sorted(result)
# 测试
set_a = {"apple", "cat", "banana", "dog", "grape"}
set_b = {"banana", "grape", "cat", "orange", "kiwi"}
print(common_long_strings(set_a, set_b))# 输出:['banana', 'grape']
```

程序逻辑解析:

找出两个集合的交集, 过滤长度大于3的字符串, 最后排序。

程序语法要点:

- 多重条件 `[s for s in common if len(s) > 3]`
- 字典序排序 `sorted(result)`

示例13: 接收一个嵌套列表, 返回所有元素的集合(展开嵌套)

```
def flatten_to_set(nested_list):
    result = set()
    def flatten(element):
        if isinstance(element, list):
            for item in element:
                flatten(item)
        else:
            if isinstance(element, (int, float, str, bool, tuple)):
                result.add(element)
    flatten(nested_list)
    return result
# 测试
print(flatten_to_set([1, [2, 3], [4, [5, 6]], 7]))# 输出:{1,2,3,4,5,6,7}
```

程序逻辑解析：

使用递归函数遍历嵌套列表的所有层级，将非列表元素添加到结果集合中，实现深度扁平化。

程序语法要点：

- **递归展开** `flatten(element)` 递归处理嵌套
 - **类型检查** `isinstance(element, list)` 判断是否继续递归
 - **元素类型过滤** 只添加基本数据类型到集合
-

示例14：找出同时选修了"数学"和"物理"的学生姓名

```
def find_math_phys_students(student_courses):
    result = []
    for student, courses in student_courses.items():
        if "数学" in courses and "物理" in courses:
            result.append(student)
    return result

# 测试
students = {
    "张三": {"数学", "语文", "英语"},
    "李四": {"数学", "物理", "化学"},
    "王五": {"物理", "生物", "历史"},
    "赵六": {"数学", "物理", "英语"}
}
print(find_math_phys_students(students))# 输出:['李四', '赵六']
```

程序逻辑解析：

遍历学生课程字典，检查每个学生是否同时包含指定课程。

程序语法要点：

- **字典遍历** `for student, courses in student_courses.items()`
 - **集合成员检查** `"数学" in courses and "物理" in courses`
-

示例15：根据操作符返回对应的集合运算结果

```
def set_operation(set1, set2, operator):
    operations = {
        "union": set1.union(set2),
        "intersection": set1.intersection(set2),
        "difference": set1.difference(set2),
        "symmetric_difference": set1.symmetric_difference(set2)
    }
    return operations.get(operator, set())

# 测试
```

```
a = {1,2,3}
b = {3,4,5}
print(set_operation(a,b,"union"))# 输出:{1,2,3,4,5}
print(set_operation(a,b,"symmetric_difference"))# 输出:{1,2,4,5}
```

程序逻辑解析：

使用字典映射操作符到对应的集合运算方法。

程序语法要点：

- 操作符映射 `operations = {"union": set1.union(set2), ...}`
- 字典get方法 `operations.get(operator, set())`

示例16：找出整数集中能表示为两个不同元素之和的整数

```
def sum_elements(s):
    elements = list(s)
    result = set()
    for i in range(len(elements)):
        for j in range(i+1, len(elements)):
            sum_val = elements[i] + elements[j]
            if sum_val in s:
                result.add(sum_val)
    return result

# 测试
print(sum_elements({1,2,3,4,5,6}))# 输出:{3,4,5,6}
```

程序逻辑解析：

检查集合中每个元素是否可以通过其他两个不同元素相加得到。

程序语法要点：

- 双重循环 遍历所有元素对组合
- 集合成员检查 `if sum_val in s`

示例17：找出相同字符异位词并分组

```
def group_anagrams(str_set):
    # 使用排序后的字符串作为键
    groups = {}
    for s in str_set:
        key = "".join(sorted(s)) # 对字符串中的字符排序，得到统一的键
        if key not in groups:
            groups[key] = []
        groups[key].append(s)
```

```

    return list(groups.values())

# 测试
print(group_anagrams({"abc", "cba", "bac", "def", "fed"}))
# 输出 [['abc', 'cba', 'bac'], ['def', 'fed']]

```

程序逻辑解析：

使用排序后的字符串作为键，将异位词分组到同一列表中。

程序语法要点：

- 排序键生成 `"".join(sorted(s))`
- 字典分组 `groups[key] = []` 和 `groups[key].append(s)`

示例18：判断一个集合可否被拆分为两个元素之和相等的子集

```

def can_split_equal_sum(s):
    total = sum(s)
    # 如果总和是奇数，不可能拆分
    if total % 2 != 0:
        return False
    target = total // 2
    elements = list(s)
    # 检查是否存在子集的和等于 target
    def find_subset(index, current_sum):
        if current_sum == target:
            return True
        if index >= len(elements) or current_sum > target:
            return False
        # 尝试包含当前元素
        if find_subset(index + 1, current_sum + elements[index]):
            return True
        # 尝试不包含当前元素
        return find_subset(index + 1, current_sum)
    return find_subset(0, 0)

# 测试
print(can_split_equal_sum({1, 2, 3, 4})) # 输出 True (1+4=2+3)
print(can_split_equal_sum({1, 2, 3, 5})) # 输出 False

```

程序逻辑解析：

使用递归回溯检查是否存在子集的和等于总和的一半。

程序语法要点：

- 递归回溯 `find_subset(index, current_sum)`
- 提前终止条件 `if current_sum == target: return True`

示例19：找出能通过拼接集合中两个不同单词形成目标字符串的组合

```
def find_word_pairs(word_set, target):
    pairs = []
    for word1 in word_set:
        if len(word1) >= len(target):
            continue
        word2 = target[len(word1):]
        if word2 in word_set and word1 != word2:
            pairs.append((word1, word2))
    return pairs

# 测试
words = {"hello", "world", "helloworld", "hi", "there", "hithere"}
print(find_word_pairs(words, "helloworld"))# 输出:[('hello', 'world')]
print(find_word_pairs(words, "hithere"))# 输出:[('hi', 'there')]
```

程序逻辑解析：

遍历集合中的单词，检查剩余部分是否也在集合中。

程序语法要点：

- 字符串切片 `target[len(word1):]`
- 集合成员检查 `word2 in word_set and word1 != word2`

字典数据处理程序实践

示例0-1：生成一副标准扑克牌的编码(花色+牌点)

```
def generate_standard_deck():
    # 定义花色：黑桃(S)、红桃(H)、方块(D)、梅花(C)
    suits = ["S", "H", "D", "C"]
    # 牌点：14(A)到2
    ranks = list(range(14, 1, -1))
    deck = []
    for suit in suits:
        for rank in ranks:
            card = f"{suit}{rank}"
            deck.append(card)
    return deck

if __name__ == "__main__":
    poker_deck = generate_standard_deck()
    # 按花色分组打印
    suit_names = {"S": "黑桃", "H": "红桃", "D": "方块", "C": "梅花"}
    for i in range(0, 52, 13):
        suit_code = poker_deck[i][0]
```



```
suit_name = suit_names[suit_code]
print(f"{suit_name}:{poker_deck[i:i + 13]}")
```

程序逻辑解析:

通过花色和点数的嵌套循环生成52张标准扑克牌，每张牌用"花色+点数"格式编码。

程序语法要点:

- **嵌套循环** `for suit in suits: for rank in ranks`
- **格式化编码** `f"{suit}{rank}"`
- **列表构建** 逐步添加生成的牌

示例0-2: 生成随机洗好的扑克牌

```
import random
def generate_random_deck():
    suits = ["S", "H", "D", "C"]
    ranks = list(range(14, 1, -1))
    deck = [f"{suit}{rank}" for suit in suits for rank in ranks]
    random.shuffle(deck)
    return deck
if __name__ == "__main__":
    random_deck = generate_random_deck()
    print("随机分布的扑克牌(共52张):")
    for i in range(0, 52, 13):
        print(random_deck[i:i + 13])
```

程序逻辑解析:

先生成有序的标准扑克牌，然后使用随机洗牌算法打乱牌序，生成随机分布的牌组。

程序语法要点:

- **随机打乱** `random.shuffle(deck)`
- **列表推导式** 简洁的牌组生成
- **分块显示** 按13张一组打印

示例0-3: 对13张牌按花色升序、同花色按点数降序排序

```
def sort_by_suit_then_rank(cards):
    # 花色排序: S(3)>H(2)>D(1)>C(0)
    suit_order = {"S": 3, "H": 2, "D": 1, "C": 0}
    # 点数排序: 14(A)到2
    rank_order = {str(rank): rank for rank in range(2, 15)}
    # 排序: 先按花色升序, 再按点数降序
    sorted_cards = sorted(cards, key=lambda x: (suit_order[x[0]], -
rank_order[x[1:])))
```

```
    return sorted_cards

if __name__ == "__main__":
    poker_cards = generate_random_deck()[13]
    print("原始牌组:", poker_cards)
    sorted_by_suit_rank = sort_by_suit_then_rank(poker_cards)
    print("按花色+点数(同色大到小):", sorted_by_suit_rank)
```

程序逻辑解析:

定义花色和点数的排序权重，使用多键排序函数先按花色升序排列，同花色内按点数降序排列。

程序语法要点:

- **多键排序** `key=lambda x: (suit_order[x[0]], -rank_order[x[1:]])`
- **字典映射** 花色和点数的权重映射
- **降序排列** 使用负号实现点数降序

示例0-4: 将扑克牌编码映射为中文名称

```
def card_code_to_name(card):
    suit_map = {"S": "黑桃", "H": "红桃", "D": "方块", "C": "梅花"}
    rank_map = {"14": "A", "13": "K", "12": "Q", "11": "J", "10": "10", "9": "9",
                "8": "8", "7": "7", "6": "6",
                "5": "5", "4": "4", "3": "3", "2": "2"}
    suit = card[0]
    rank = card[1:]
    suit_name = suit_map[suit]
    rank_name = rank_map.get(rank, rank)
    return f"{suit_name}{rank_name}"

# 测试(随机生成13张牌并显示名称)
random_cards = generate_random_deck()[13]
for i, card in enumerate(random_cards, 1):
    print(f"{i}. {card_code_to_name(card)}")
```

程序逻辑解析:

通过字典映射将扑克牌编码转换为中文名称，分别处理花色和点数的映射关系。

程序语法要点:

- **字典映射** `suit_map` 和 `rank_map`
- **字符串拼接** `f"{suit_name}{rank_name}"`
- **切片操作** `card[0]` 和 `card[1:]` 分离花色和点数

示例0-5: 根据用户输入显示对应的扑克牌(简化版, 无图像显示)

```
def show_poker(card_input):
    # 验证输入格式
    if len(card_input) < 2:
        print("输入格式错误! 示例: S2(黑桃2)、H3(红桃3)")
        return
    # 解析花色和数字
    suit = card_input[0].upper()
    num_str = card_input[1:]
    # 验证数字
    try:
        number = int(num_str)
    except ValueError:

        print("数字部分必须是整数! ")
        return
    # 验证花色合法性
    valid_suits = ['S', 'H', 'C', 'D']
    if suit not in valid_suits:
        print(f"花色错误! 支持的花色: {valid_suits} (S-黑桃、H-红桃、C-梅花、D-方块)")
        return
    # 验证数字范围
    if number < 2 or number > 14:
        print("数字范围错误! 支持2-14(A)")
        return
    # 显示牌名
    card_name = card_code_to_name(f"{suit}{number}")
    print(f"你选择的牌是: {card_name}")

# 测试
show_poker("S14") # 输出:你选择的牌是: 黑桃A
```

程序逻辑解析:

验证用户输入的扑克牌编码格式, 检查花色和点数的合法性, 最终显示对应的中文牌名。

程序语法要点:

- **输入验证** 多层次的格式和范围检查
 - **错误处理** 详细的错误提示信息
 - **大小写处理** `card_input[0].upper()` 统一花色格式
-